Report File

Lubin Venter
Sebastian Snyman

Index

Introduction

Literature Review

Gases such as butane, propane, and LPG (a mixture of propane and butane) are commonly used in household gas stoves. Carbon monoxide is produced through incomplete combustion in gas appliances. In their pure form, these gases are odorless, colorless, and invisible, making them extremely difficult to detect without a gas detector. If inhaled in large amounts, any of these gases may lead to symptoms ranging from headaches to fatal outcomes, depending on the level of exposure.

Although these gases are often treated with chemical odorants to give them a distinct smell for easier detection, problems arise if residents are in another room and cannot smell the leak—or if they suffer from anosmia (a condition causing the inability to smell). In such cases, exposure could result in avoidable harm had a gas detector been present.

Store-bought gas detectors are typically expensive and capable of detecting only a limited range of gases. Many of them require a constant power supply, which renders them ineffective during a power outage.
This poses a significant risk—particularly in countries where gas-operated appliances such as stoves are widely used and power outages are common. If a leak occurs in a poorly ventilated room, the danger to residents can escalate rapidly.

The fact that some of these store-bought gas detectors only work when they have a constant power supply is especially dangerous in countries where power outages are frequent. South Africa is a great example of this type of country, where load shedding frequently happens and leaves people without power for sometimes long periods of time.

Back on the other fact that store-bought gas detectors are expensive, a consumer grade multi-gas detector (detects a few gasses simultaneously) for home use goes for around R2 500 - R5000. This is a large amount and can especially be a difficult amount to pay off in a third world country. Our gas detector can detect multiple gasses, has a built in battery and alarm, and only costs R905,29 to produce (if the equipment used (soldering set) is not included in the price) making it cheaper than store-bought gas detectors. The other option is getting a basic single-gas detector that only detects one type of gas, but is cheaper. This is a risky choice as the battery could run out, and what if a gas is leaking that the detector can not detect?

This gap in available solutions has motivated the development of a gas detector that can identify all common household gases, remain operational during a power outage, and be sold at an affordable price—addressing all of these critical needs.

Need Or Problem Defined

In South Africa, load shedding presents a major challenge for residents. The issue is compounded by the fact that 77% of electrified households use electric stoves for cooking. As a result, many South Africans have begun turning to gas as an alternative fuel source.

Commonly used gases in household stoves include propane, butane and LPG. In addition, carbon monoxide remains a relevant concern due to its presence in emissions from gas appliances. These gases are difficult to detect without a gas detector and pose a serious health risk.

Commercial gas detectors are often expensive, limited in detection capabilities, and dependent on mains electricity. This becomes particularly dangerous in areas that rely on gas appliances and experience frequent power outages.

As such, there is a clear need for a low-cost, multi-gas detection system that will remain functional during power failures through the use of a backup power supply.

Aim

This project aims to create a gas detector that can be installed near gas appliances to detect leaks from a variety of gases, including propane, butane, LPG, and carbon monoxide. The gas detector will be affordable, accurate, and capable of functioning during electricity outages. It will be designed to immediately and effectively alert residents if a gas leak occurs through a built in buzzer that makes a distinct alarm sound.

Engineering Goals/Design Goals/Algorithms

- The gas detector must be affordable (In the R500-R1500 price range).
- The gas detector must detect multiple gasses, ranging from propane, butane, LPG, and carbon monoxide.
- The gas detector must be able to function in power outages.
- The gas detector must be able to effectively alert residents of a gas leak.
- The gas detector must be able to withstand some heat, as it might be placed near a stove with open flames.
- Final prototype must be able to send a notification to your phone if a gas leak is detected

Method

Materials

The following components will be required to construct the gas detector(subject to change):

- Gas Sensors (MQ-9) – To detect carbon monoxide and flammable gases.
- Microcontroller (ESP32) – Supports MicroPython and provides processing power.
- LCD Display (2.8" Capacitive Touch) – For user interface and gas level readouts.
- Buzzer or LED Indicator – To trigger visual or audible alerts in unsafe conditions.
- Water-Resistant Enclosure – To protect components from humidity or splashes.
- Power Supply (12V DC Adapter) – For stable voltage from mains electricity.
- Rechargeable Battery (12V Li-ion or LiPo) – Backup power during outages.
- Battery Management Module (3.7V LiPo Charger with DC-DC Boost) – For regulated charging and power delivery.
- Voltage Regulator (Buck Converter) – To reduce and stabilize voltage output.
- DC Barrel Jack (2.1mm / 5.5mm) – For connecting standard adapters to the power circuit.
- JST Connectors – For compact and reliable connections between modules.
- Battery Connector Kit (50A or 120A) – For quick-connect backup battery integration.
- DC Jack to Alligator Clips – For flexible testing with variable power sources.
- Standard Electrical Wires (18–22 AWG and 14–16 AWG) – To distribute power effectively depending on current demand.

Not all of these parts were used in the making of the gas detector, the true parts list follows on the next page:

Final List Of Materials:

- Battery LiPo 1000 mAh 7.4V
  - https://www.robotics.org.za/603450?search=lipo%20battery
- MQ 5 Toxic Gas Sensor:
  - https://www.winsen-sensor.com/d/files/MQ-5.pdf
- Robotico ESP32 Development Board 2.4GHz Dual-Core WiFi + Bluetooth
  - https://www.takealot.com/robotico-esp32-development-board-2-4ghz-dual-core-wifi-bluetooth/PLID72203100
  - Cheap microcontroller with a big variety of features. Will act as the core/controller for the gas detector.
- Buck Converter [BMT ADJ DC/DC MODULE 3A+DISPLAY]:
  - https://www.communica.co.za/products/bmt-adj-dc-dc-module-3a-display
  - Can input a larger voltage and output a smaller adjustable voltage. This is used to step down the 12V from the battery to 5V for the microcontroller.
- MF 1/4W 1% E12 RES KIT
  - https://www.communica.co.za/products/mf-1-4w-1-e12-res-kit?variant=31251775357001
  - A resistor kit which contains the required resistors to make a voltage divider.
- Multipurpose 12V DC 2AMP Power Supply - AC 240V
  - https://www.takealot.com/multipurpose-12v-dc-2amp-power-supply-ac-240v/PLID46906813?gad_campaignid=20213916716&gad_source=1&gclid=Cj0KCQjw5JXFBhCrARIsAL1ckPu2BIxR7SLa2u_vU9od8uVXzJ7s0YKz-su71M2onFX_yjZEbW4BEp0aAk3QEALw_wcB&gclsrc=aw.ds
- DC Barrel Power Jack, Female, SMD, 2155 Standard, 4 Pack
  - https://www.robotics.org.za/connectors/dc-jack-connectors/DC050-2155-SMD
- HKD BREADBOARD 8,3X5,5CM 400TP
  - https://www.communica.co.za/products/hkd-breadboard-8-3x5-5cm-400tp?variant=31932627419209
  - Allows for more convenient connection of components.
- USB CABLE 1,5M AM-MICRO
  - https://www.communica.co.za/products/usb-cable-1-5m-am-micro?variant=17185748779081
  - Used to input 5V to the microcontroller via its usb 2.0 compatible port.
- HKD ELECTRONIC BUZZER
  - https://www.communica.co.za/products/hkd-electronic-buzzer?variant=47729630839084
  - Buzzer that can work with both 3.3V and 5V.

- HKD RIBBON JUMPER 40W M/M 30CM
  - https://www.communica.co.za/products/hkd-ribbon-jumper-40w-m-m-30cm?variant=31916109561929
  - Wires used to make connections.
- HKD BREADBOARD JUMPER KIT (140)
  - https://www.communica.co.za/products/hkd-breadboard-jumper-kit-140?variant=31932617097289
  - More wires to connect components.
- A Soldering Set
  - Used to make connections more secure via soldering the pins of components to wires.
  - Already have a soldering set.
  - https://www.takealot.com/16-piece-soldering-kit-with-meter-and-stand-adjustable-heat-200-/PLID91983733?srsltid=AfmBOopdpGcytro52_7AHngCR2IIXgmtrVREhXElgu_z7g_c_tnUN4bMB9U
- Plastic Tubing used to make the wiring cleaner and more clear to see what its purpose is. Black is used to show it is for ground, Red for VCC and Blue for control or input and output.
  - Supplied by Riaan Snyman
- ABS Enclosure 115 x 75 X 52 with Mounting Tags, Grey
  - https://www.robotics.org.za/A3-TAG-GREY?search=abs%20enclosure%20115
- Universal 2A LiPo Charger, 4.2V / 7.4
  - https://www.robotics.org.za/TP5100?search=Lithium%20Polymer%20charger

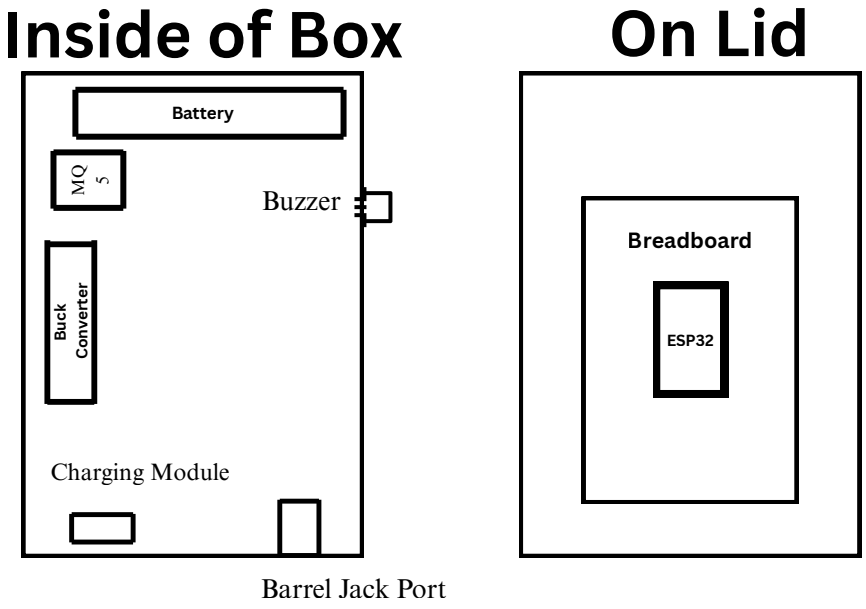
Procedure

The gas detecctor was first turned on. Then after waiting at least an hour, the gas sensors are ready to use to take data. 100 values are taken. These values are the output from the microcontroller that read the analog pins from the sensors and were converted to voltage. Each value was read every second. Thus testing took 100 seconds each. For the first 30 seconds no gas or change in environment was introduced. Between 30 and 40 seconds the gas was introduced or the mechanism releasing it was turned on. After the 40 second mark the gas introduced is taken away or the mechanism is turned off. The following values are then measured till the 100 second mark.

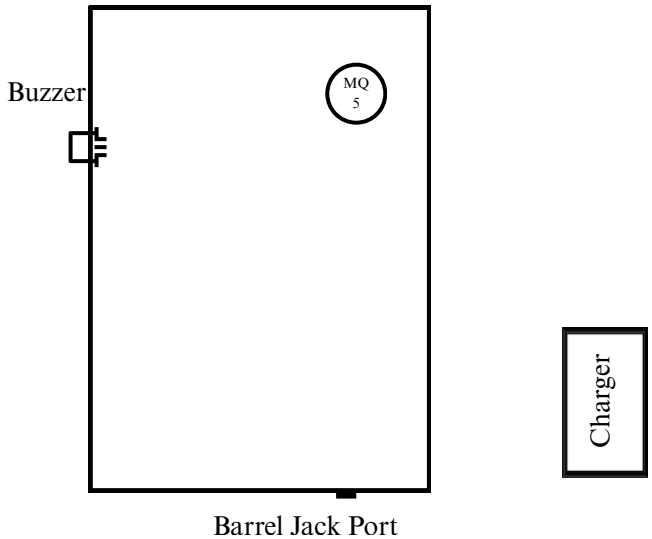
Engineering Method

Planning

Physical Prototype:



Inside Final Physical Prototype



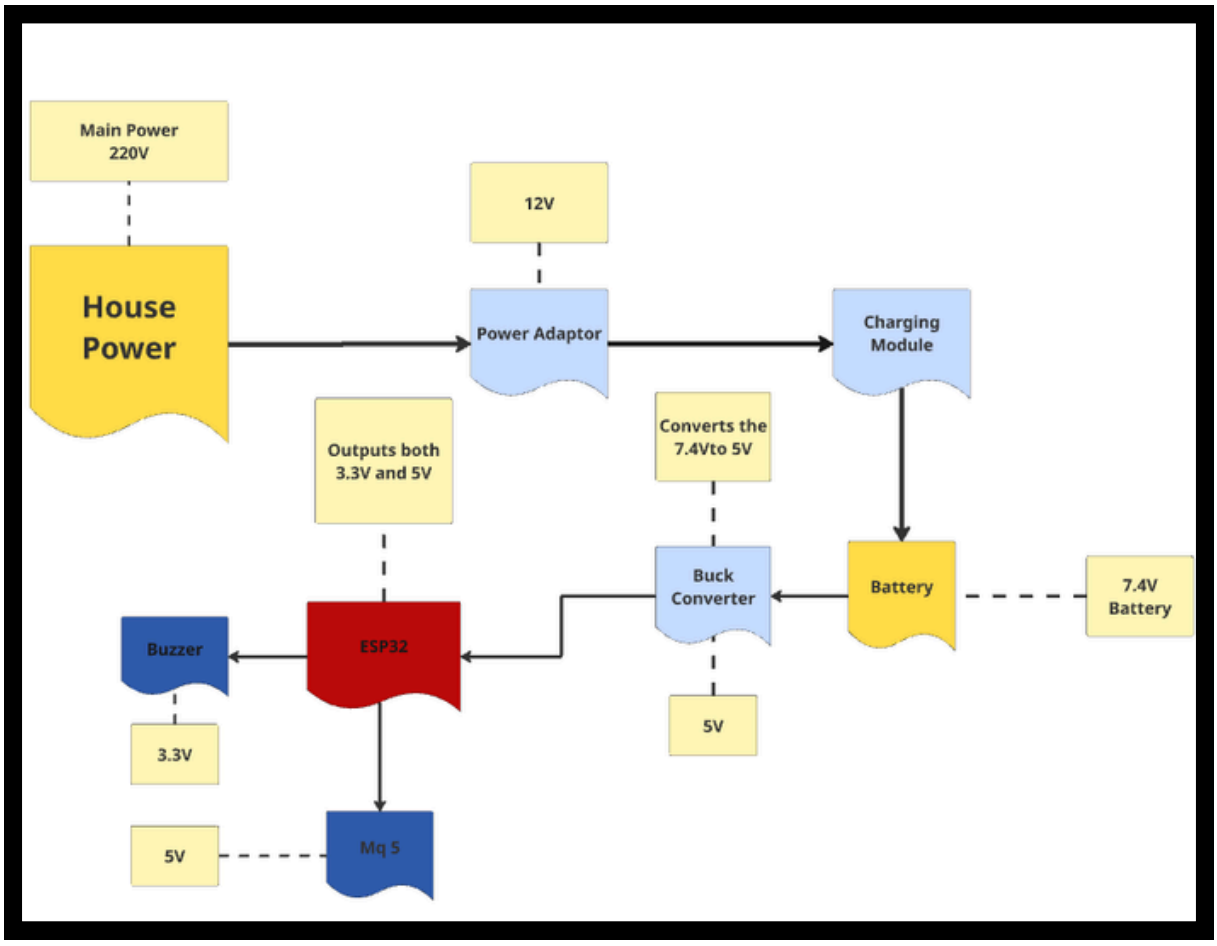
Front Final Physical Prototype

Components Sizes:

Component	Length (mm)	Width (mm)	Height (mm)
Casing	115	75	*52*
Battery	51.5	34	10
Charging Module	25	17.3	5
Power Adaptor	70	50	30
ESP32	50	25	14
Buck Converter	65	47	10
MQ 5	32	20	22
Buzzer	33	13	9
Breadboard	83	55	9
Barrwl Jack Port	14.8	13	11

A power diagram was made to show the how the circuit handled going down in voltage from the house power to components needing 5V and 3.3V. The diagram show that the 7.4V battery is powered via the charging module - TP5100 - which converted 220V to ±8.4V via a power adaptor to power the battery.(Battery LiPo 1000 mAh 7.4V) Then using the buck converter to step down the voltage from 7.4V to 5V the ESP32 microcontroller is powered via its usb port. The microcontroller is then able to power 5V or 3.3V to the relevant components.

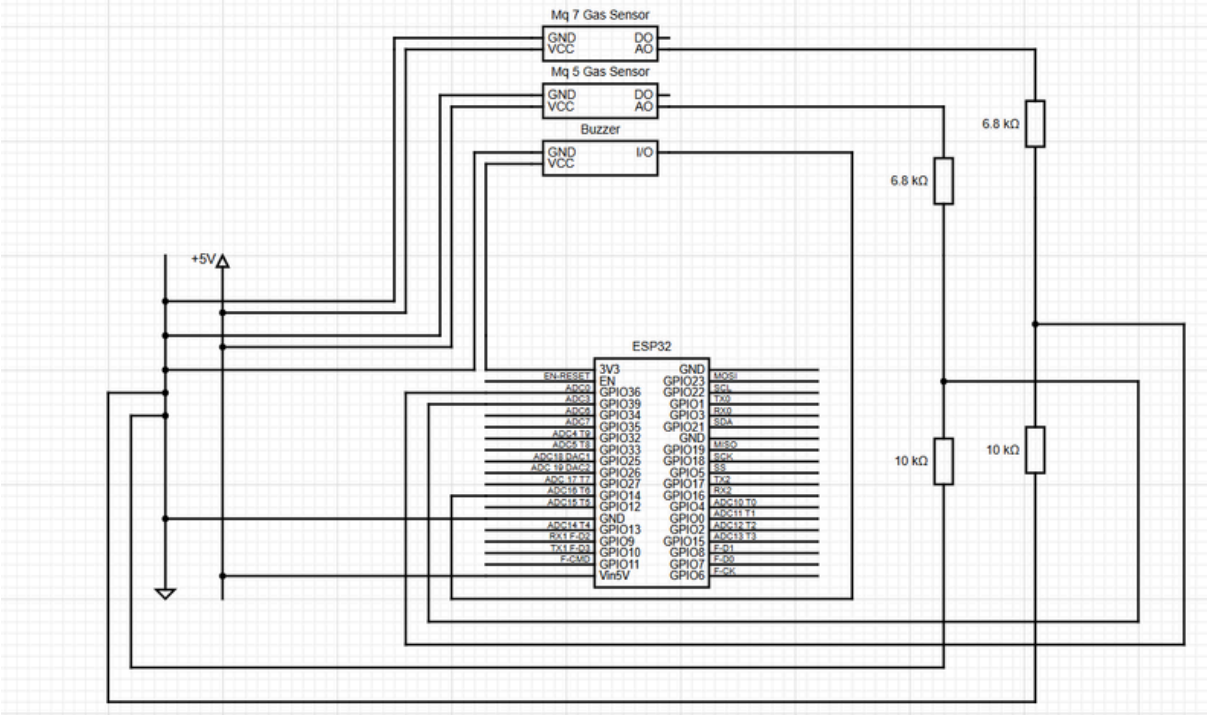
Power Diagram:



Wiring Diagram:
This diagram was made for the first prototype, and is mostly still relevant for the final prototype as the only difference is that the MQ7 gas detector and its voltage divider is no longer present in the system.

Diagram that shows the important wiring between pins of components and how they are powered. The following will describe the functions of the different pins/components:

- 5V Rail
  - The rail that gives the 5V positive terminal for components.
  - Is powered via the Vin5V pin of the ESP32.
  - Connects to VCC pins.
- GND Rail
  - Gives ground/negative for the whole circuit.
  - 0V
- AO
  - Sends electrical signal to GPIO pins so the microcontroller can read them and convert the reading to voltage.



Creating

The first prototype was built with one gas sensor to test the microcontroller, its programming/code made use of phase one through 4 (see Appendix) and was tested to see if it could read the analog output correctly. Then it was further developed to operate with both sensors and then finally with the buzzer. As a safety measure voltages and current were constantly checked for with a multimeter so that components would not break or fry. The buzzer was also first tested to see if it did work. The final version of the first prototype consisted of the shoebox that was painted white to present the project better. The diagrams were also put on the box.

Here is how the components were connected:

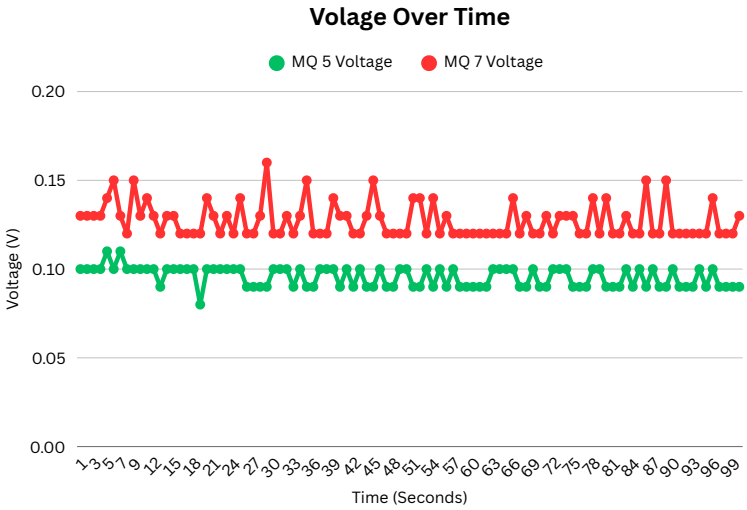
The half of the ESP 32 was connected to the breadboard. Then the Vin5V was connected to the positive power rail. Then the GND pin was connected to the negative rail. Now the two gas sensors' VCC pins were connected to the positive power rail and the GND pins to the negative power rail. The buzzer's VCC pin was connected to the 3V3 pin of the ESP32 pin to power it and the GND pin to the negative power rail. The voltage dividers was then made for the two sensors.(6.8kΩ resistor and 10kΩ resistor each) The 6.8kΩ resistors were connected to the the AO pins first, then the 10kΩ resistors were connected to the 6.8kΩ resistors. The negative power rail was then connected to the 6.8kΩ resistors. The ADC pins were then connected to the point in between the two resistors for each sensor. This stepped down the maximum voltage. The I/O pin of the buzzer was then connected to a GPIO pin that would control it going on or off. The wiring was then made cleaner with tubing that was used to insulate the wires and pins. The tubing is also colour coded; blue for control, red for positive and black for negative. Then the components were put in the shoebox. This is where the battery and buck converter were also placed. The computer uploaded the final program to the microcontroller and thereafter everything was connected. The battery to the charger module. The battery to the buck converter and the buck converter to the ESP32 with a USB adapter to the port of the microcontroller. The prototype was then finished and ready. The final (Phase 6 Coding mentioned in Appendix) prototype used almost the same steps for building it, but it was fitted in a neater, smaller container, only used one gas sensor (MQ5) instead of two (MQ5 and MQ7), as the MQ7 was deemed as unnecessary. This final prototype made use of the phase 6 code (See Appendix). It also featured a new battery which is more space efficient, and a new charging module and charging port. The charging port requires a 12V DC power supply to be plugged into the new charging module, which also converts the 12V to 8,4V, which then goes to the battery. The battery gives off 7.4V to the buck converter, which takes the 7.4V down to 5V which is then used in the rest of the system. This makes the final prototype more space efficient, cost efficient and simplifies the charger needed to charge the gas detector. This final prototype also includes a feature to send out a distress message via the Blynk app to alert residents of a possible gas leak.

Results Of Testing

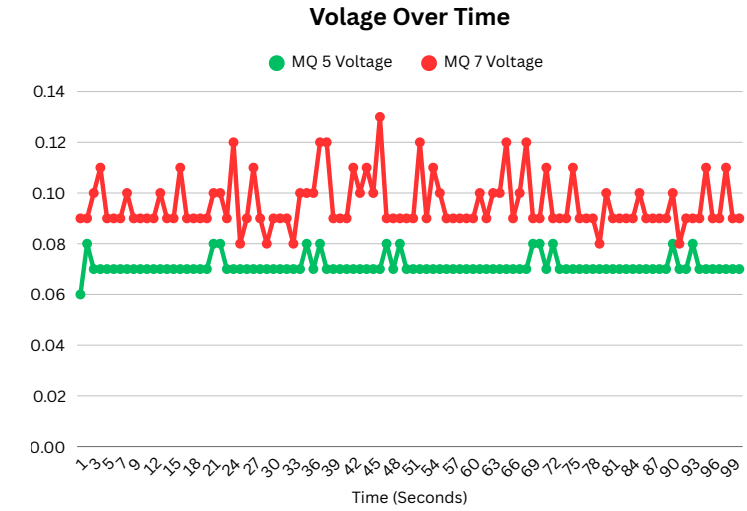
(Tests were done with the first prototype, as the final prototype still uses the same coding and gas detectors, so new results were not needed.)

Natural Air (Control)

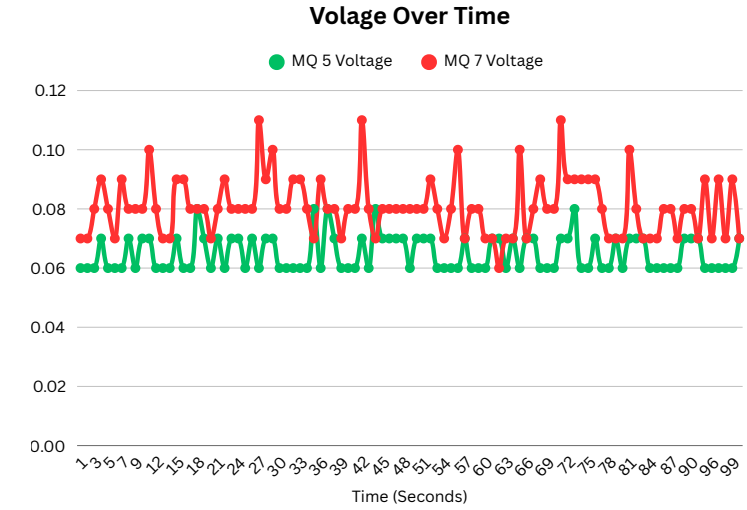
Test 1



Test 2

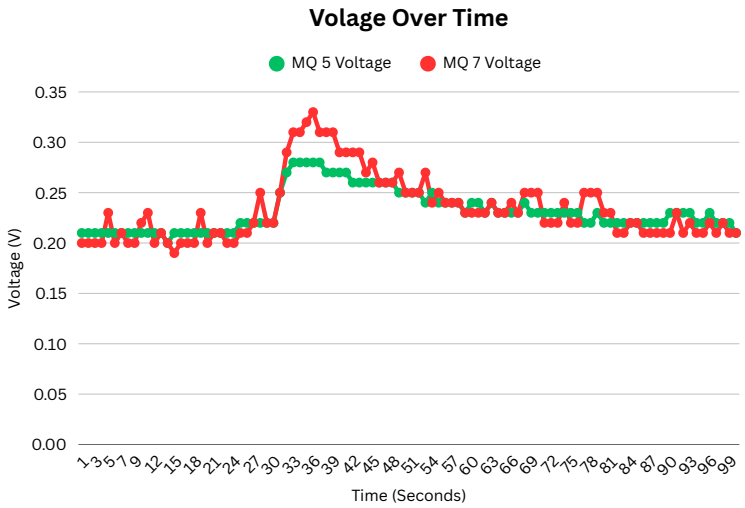


Test 3

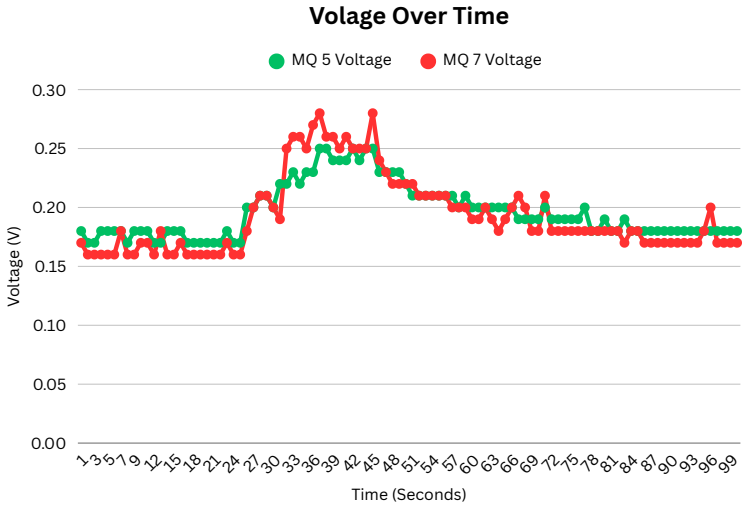


Candle
(CO was tested)

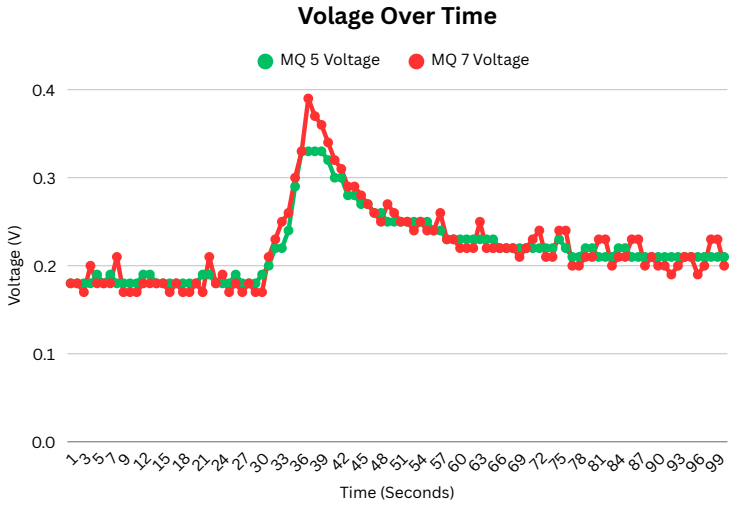
Test 1



Test 2

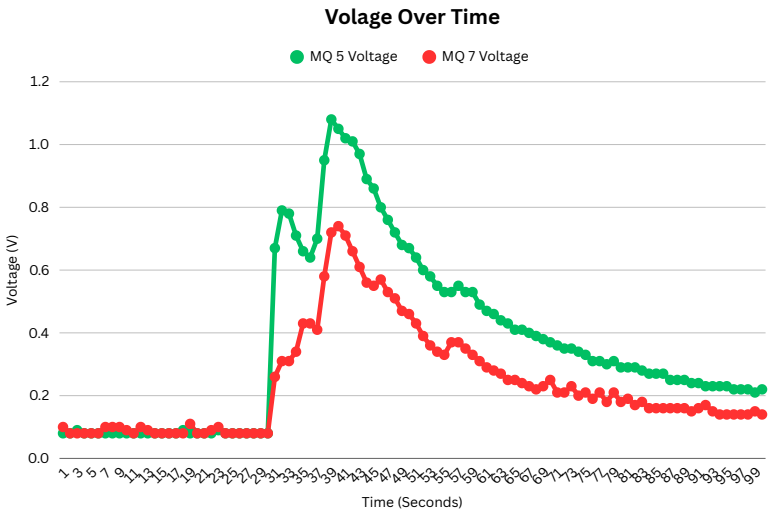


Test 3

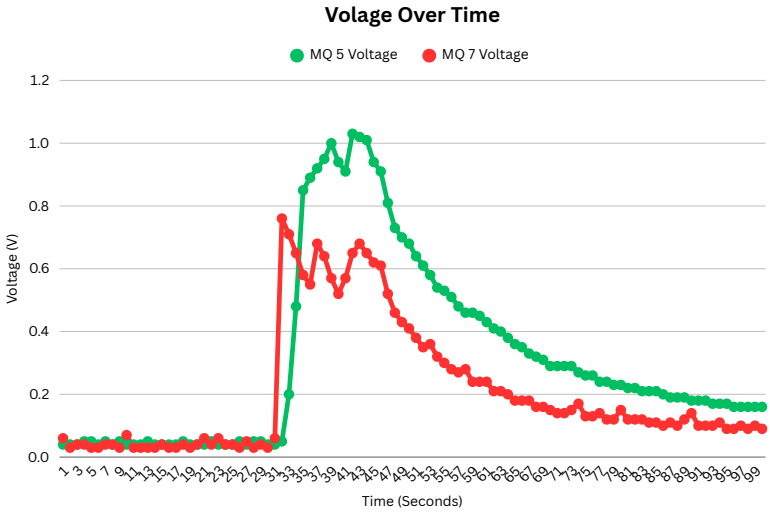


Lighter
(Butane was tested)

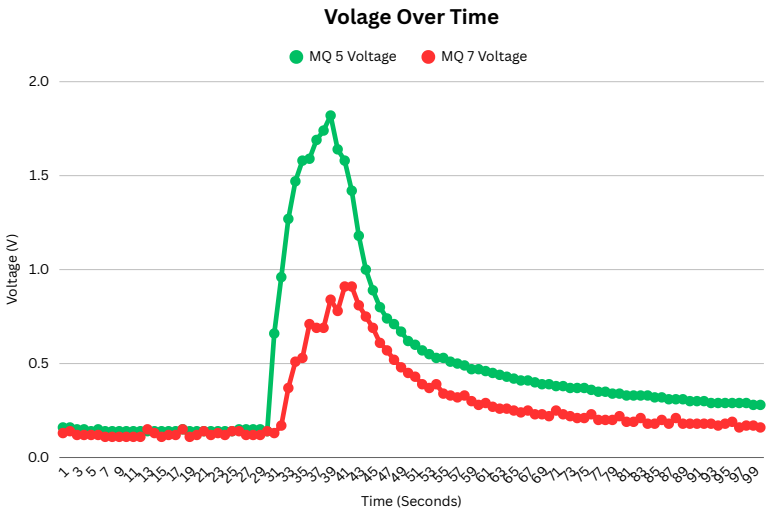
Test 1



Test 2

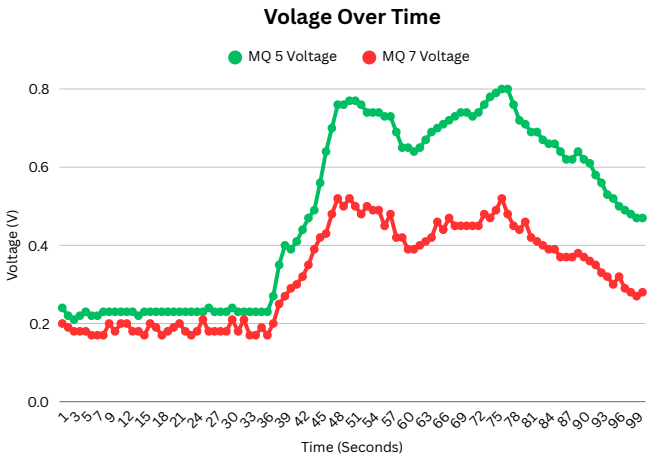


Test 3

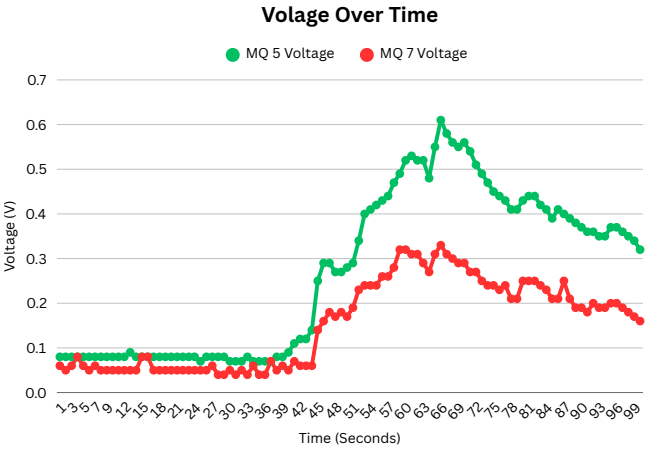


Gas Stove
(LPG was tested)

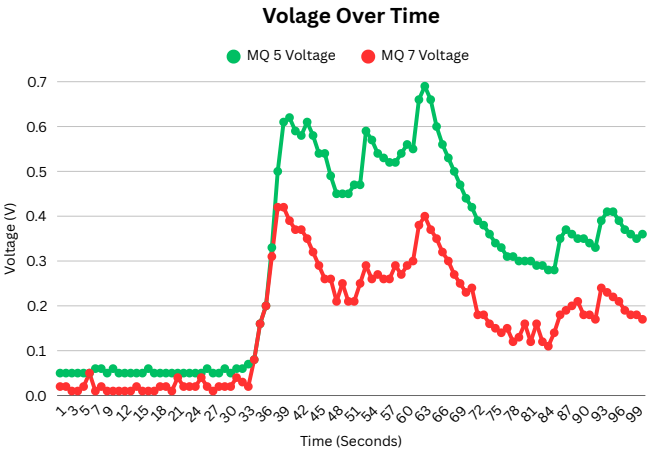
Test 1



Test 2

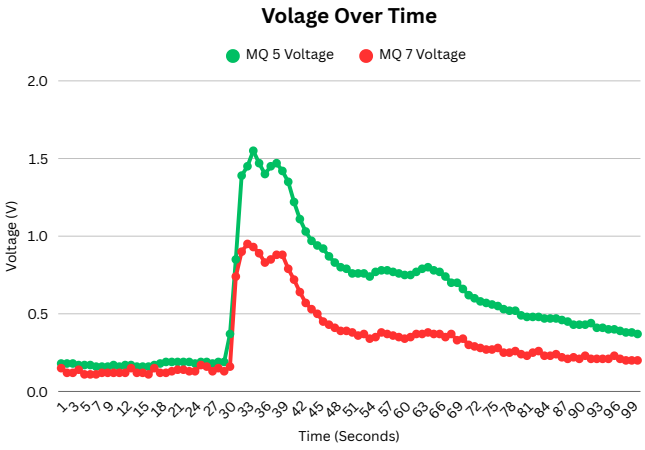


Test 3

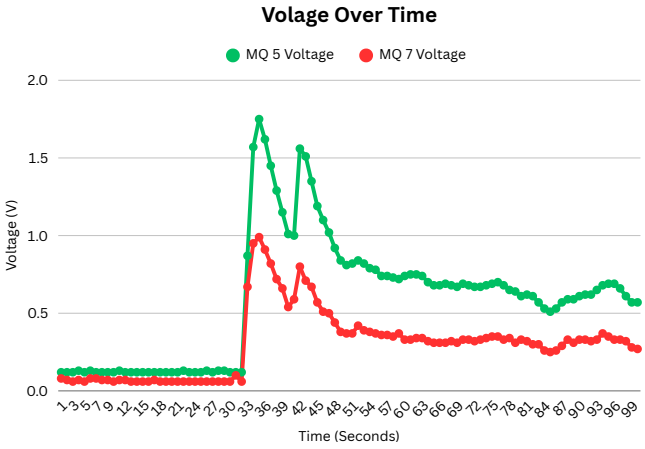


Gas Lighter
(LPG was tested)

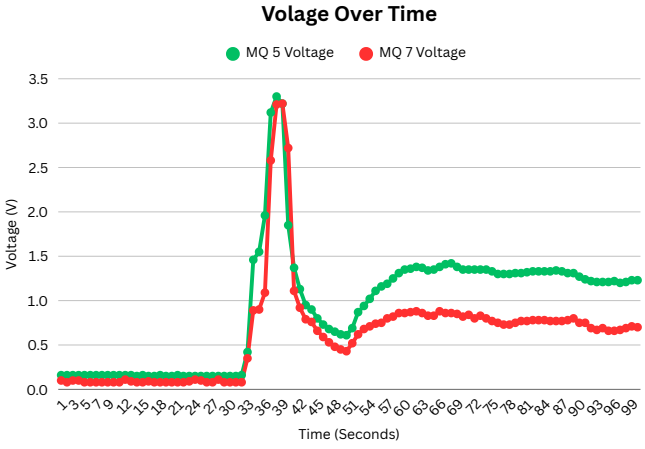
Test 1



Test 2

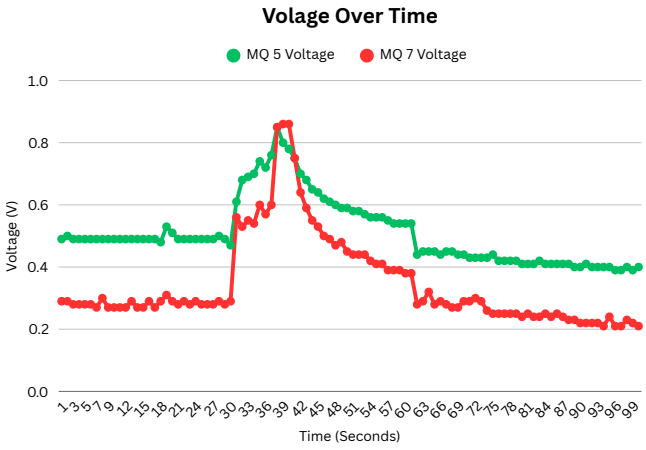


Test 3

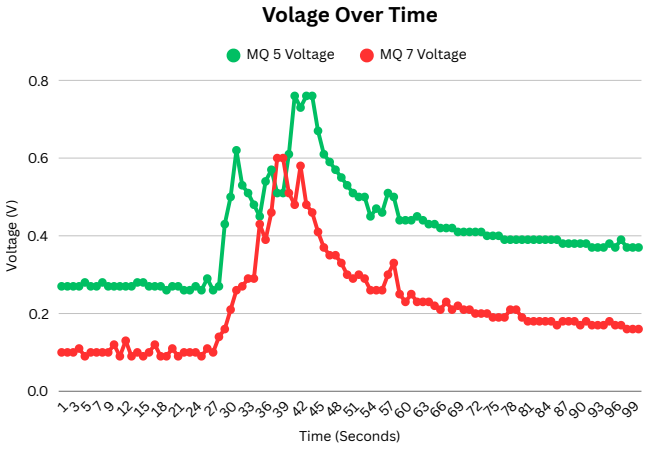


Wood Smoke
(CO was tested)

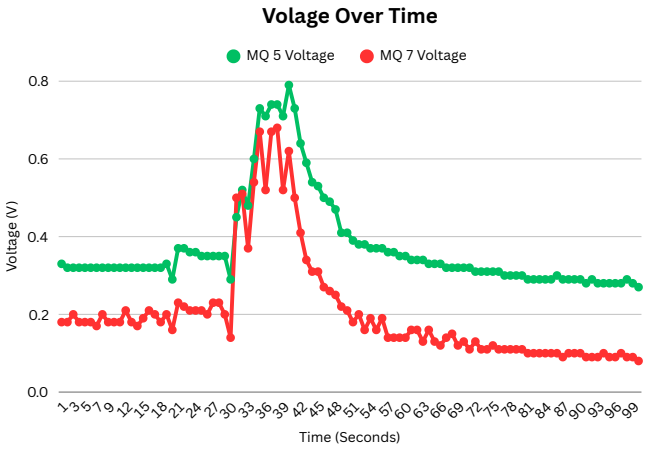
Test 1



Test 2



Test 3



It was found that the first prototype gas detector can run for roughly 16 hours without powering the battery (let it run overnight numerous times till the next day numerous times), this was not tested extremely accurate but it deems it acceptable for the purpose of the gas detector being able to run without power for at least four hours. The buzzer was buzzing every time the threshold of 0.7V was reached with the sensors. The final prototype gas detector's battery can power the system for around six and a half hours. This is a large drop in the number of hours that the battery can power the system, when compared to the first prototype, but the cost cuts made because of the new battery makes it worth it, as the new prototype's battery took the price down by about R1000. The six and a half hour battery is deemed acceptable for this project.

Results

When gasses were introduced the voltages that were read spiked. The MQ 5 seemed to be able to detect all the gasses the MQ 7 could detect and more effectively. CO was generally more minimal when it comes to detecting it via wood smoke or candle smoke. The LPG gasses were detected with high voltages. The gas stove seemed to release more gas in the air compared to other tests. When the fire was made the gas detector already detected the smoke in the room thus the readings started at a high voltage compared to the reading of natural air. The buzzer did buzz every time the voltage surpassed 0.7V.

Discussion

The gas detector and its sensors clearly sensed when there was a change in environment and when a gas was introduced. When the gas was introduced there was always a spike in voltage for both sensors. Certain gasses/tests took a few more seconds for the voltages to spike, this is because of the gas still needing to rise to the sensors. When the gas was taken away the sensors gradually sensed less of the gas in the air for the voltage gradually decreased. Smoke had minimal amount of CO to detect. The starting values of the smoke tests were a bit higher because of the smoke in the air being already detected before being introduced to the sensors. The MQ 5 senses the same gasses as the MQ 7 and more efficiently. It generally had more sensitivity for all the harmful gasses. The gas detector is completely able to detect the harmful gasses it needs to detect in households and alert those in the house.

Limitations and errors

The gas detector could not be compared with another one made by others in the commercial market which makes it hard to compare the efficiency of the gas detector and if it senses the gasses in an effective way. An accurate gas gun could not be used (for economic reasons) to accurately test the sensors in the sense of comparing the results with the amount of gas released. The readings were not converted to parts per million which would be more user friendly. The air flow could not be completely controlled.

Recommendations for Future Research

- Researching a way to make the gas detector waterproof
- Finding cheaper alternative parts, especially for the battery and charger.
- Finding a way to convert the method of measurement of gas to parts per million.
- Researching different gas detecting components that can detect gas leaks from further away.
- Researching ways to place components more efficiently inside the box to make the gas detector smaller.
- Finding parts that consume less power.
- Researching different gas detecting sensors that can accurately display what type of gas is being detected.
- Making the connection of components as simple as possible to make it as easy as possible to recreate or mass produce the gas detector.

Conclusion

Both prototype gas detectors were completely able to detect the harmful gasses. The first prototype could run for more than 16 hours without power from the house. The final prototype could run for around 6 and a half hours without power. This means that they will be able to help South Africans during power outages and loadshedding. The gas sensors will alert users when the analog pins send over 0.7V. This ensures households who use gas stoves and other gasses are more safe from these harmful gasses. It was also found that the need for two gas sensors (MQ 5 and MQ 7) was unnecessary, thus in the final prototype the MQ 7 was removed. The battery was also switched for a more cost efficient one that was easier to find, and the charger was made more universal to help with implementing the gas detector at home. These changes made the second (final) prototype much more cost effective and by the use of a smaller case, also take up less space. As an extra precaution, the final prototype could send out a warning message to users' phones through an application called Blynk via WiFi. However improvements can be made with integrating more user friendliness and testing the prototypes more accurately. The gas detectors can successfully detect the harmful gasses and alert users with its buzzer.

Acknowledgements

Riaan Snyman - Helped with the building of the gas detector, as well as funding valuable equipment used in the building of the gas detector.

Michael Nigrini - Helped with research regarding how to build the gas detector as well as sharing knowledge on what parts are best fit for the gas detector.

Microsoft Copilot - Helped with research, grammar and spelling checks.

References

Microsoft Copilot. (2025). Grammar and spelling assistance provided via AI conversation. [online] Microsoft. Available at: https://copilot.microsoft.com/chats/BZXY92w5NY7SPx5CwKGNW

Statistics South Africa. (2024). *The state of South African households in 2023*. [online] Available at: https://www.statssa.gov.za/?p=17283

Candido, R. (n.d.). Arduino With Python: How to Get Started. [online] Real Python. Available at: https://realpython.com/arduino-python/

Miro. (2025). Miro: Innovation Workspace. [online] Available at: https://miro.com/app/dashboard/innovation-workspace/

Circuit Diagram. (2025). Circuit Diagram Web Editor. [online] Available at: https://www.circuit-diagram.org/editor/

the_3d6. (2017). Arduino CO Monitor Using MQ-7 Sensor. [online] Instructables. Available at: https://www.instructables.com/Arduino-CO-Monitor-Using-MQ-7-Sensor/

MQ7 Gas Sensor Datasheet
https://cdn.sparkfun.com/assets/b/b/b/3/4/MQ-7.pdf

MQ5 Gas Senor Datasheet
https://www.winsen-sensor.com/d/files/MQ-5.pdf

Used This Site To Learn Coding And Work With ESP32
https://randomnerdtutorials.com/

Appendix

Both prototypes used the same code, as they both still had the same sensors.

Phase 1

Test MQ 7

```
const int analogPin = 36;

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);
  delay(1000);
}

void loop() {
  int rawValue = analogRead(analogPin);  // Read ADC value
  float voltage = (rawValue / 4095.0) * 3.3;  // Convert to voltage

  Serial.print("ADC Value: ");
  Serial.print(rawValue);
  Serial.print(" | Voltage: ");
  Serial.print(voltage, 2);
  Serial.println(" V");

  delay(1000);  // Wait 1 second
}
```

Phase 2

Test MQ 7 With Buzzer

```
const int analogPin = 36;   // GPIO36 = ADC1_CH0 (for MQ-7 sensor)
const int buzzerPin = 14;   // GPIO14 = ADC2_CH6 (for buzzer)
const int threshold = 100;  // Adjust this based on testing

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);      // Set ADC resolution to 12-bit (0-4095)
  pinMode(buzzerPin, OUTPUT);    // Set GPIO14 as output for buzzer
  digitalWrite(buzzerPin, LOW);  // Make sure buzzer is OFF initially
  delay(1000);
}

void loop() {
  int rawValue = analogRead(analogPin);       // Read sensor
  float voltage = (rawValue / 4095.0) * 3.3;  // Convert to voltage

  Serial.println("MQ 7 Gas Sensor");
  Serial.print("ADC Value: ");
  Serial.print(rawValue);
  Serial.print(" | Voltage: ");
  Serial.print(voltage, 2);
  Serial.println(" V");

  if (rawValue > threshold) {
    digitalWrite(buzzerPin, LOW);  // Turn buzzer ON
    delay(10000);
  } else {
    digitalWrite(buzzerPin, HIGH);  // Turn buzzer OFF
  }

  delay(1000);  // Sample every 1 second
}
```

Phase 3

Test Whole System

```
const int mq7Pin = 36;
const int mq5Pin = 39;
const int buzzerPin = 14;

// Adjust thresholds after clean air testing
const int mq7Threshold = 1000;
const int mq5Threshold = 1000;

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);       // 12-bit ADC resolution (0–4095)
  pinMode(buzzerPin, OUTPUT);
  digitalWrite(buzzerPin, HIGH);  // OFF initially (active-low)
  delay(2000);
}

void loop() {
  int mq7Value = analogRead(mq7Pin);
  float mq7Voltage = (mq7Value / 4095.0) * 3.3;

  int mq5Value = analogRead(mq5Pin);
  float mq5Voltage = (mq5Value / 4095.0) * 3.3;

  Serial.println("---- Gas Sensor Readings ----");
  Serial.print("MQ-7 ADC: ");
  Serial.print(mq7Value);
  Serial.print(" | Voltage: ");
  Serial.print(mq7Voltage, 2);
  Serial.println(" V");

  Serial.print("MQ-5 ADC: ");
  Serial.print(mq5Value);
  Serial.print(" | Voltage: ");
  Serial.print(mq5Voltage, 2);
  Serial.println(" V");

  if (mq7Value > mq7Threshold || mq5Value > mq5Threshold) {
    digitalWrite(buzzerPin, LOW);  // Turn buzzer ON (active-low)
  } else {
    digitalWrite(buzzerPin, HIGH); // Turn buzzer OFF
  }

  delay(1000); // Sample every 1 second
}
```

Phase 4

Test Results Code

```
const int mq7Pin = 36;  // MQ-7 analog output to GPIO36
const int mq5Pin = 38;  // MQ-5 analog output to GPIO39
const int buzzerPin = 14;
int second = 0;  // Active-low buzzer on GPIO14

// Adjust thresholds after clean air calibration
const float mq7Threshold = 0.7;
const float mq5Threshold = 0.7;

const float espMaxVoltage = (5.0 / 16800.0) * 10000.0;            //the max voltage for the output in the voltage divider
const float voltageDividerMultiplier = (10000.0 + 6800.0) / 10000.0;  // Ration of esp voltage to ao voltage is 1:1.68 (16800/10000)

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);  // 12-bit ADC (0–4095)
  pinMode(buzzerPin, OUTPUT);
  digitalWrite(buzzerPin, HIGH);  // OFF initially (active-low)
  delay(2000);
}

void loop() {
  float mq7RawValue = (analogRead(mq7Pin) / 4095.0) * espMaxVoltage;
  float mq7EspVoltage = mq7RawValue;
  float mq7AoVoltage = mq7RawValue * voltageDividerMultiplier;

  float mq5RawValue = (analogRead(mq5Pin) / 4095.0) * espMaxVoltage;
  float mq5EspVoltage = mq5RawValue;
  float mq5AoVoltage = mq5RawValue * voltageDividerMultiplier;

  second = second + 1;
  Serial.print(second);
  Serial.print(",");
  Serial.print(mq5AoVoltage);
  Serial.print(",");
  Serial.println(mq7AoVoltage);

  // Buzzer alarm logic (active-low)
  if (mq7AoVoltage > mq7Threshold || mq5AoVoltage > mq5Threshold) {
    digitalWrite(buzzerPin, LOW);  // ON
  } else {
    digitalWrite(buzzerPin, HIGH);  // OFF
  }

  delay(1000);
}
```

Phase 5

```
const int mq7Pin = 36;    // MQ-7 analog output to GPIO36
const int mq5Pin = 39;    // MQ-5 analog output to GPIO39
const int buzzerPin = 14; // Active-low buzzer on GPIO14

//If these voltages are reached the buzzer is turned on.
const float mq7Threshold = 0.7;
const float mq5Threshold = 0.7;

const float espMaxVoltage = (5.0 / 10800.0) * 10000.0;            //the max voltage for the output in the voltage divider : 2.970..
const float voltageDividerMultiplier = (10000.0 + 6800.0) / 10000.0;  // Ration of esp voltage to ao voltage is 1:1.68 (16800/10000)

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);  // 12-bit ADC (0–4095)
  pinMode(buzzerPin, OUTPUT);
  digitalWrite(buzzerPin, HIGH);  // OFF initially (active-low)
  delay(2000);
}

void loop() {
  float mq7RawVoltage = (analogRead(mq7Pin) / 4095.0) * espMaxVoltage; //to get it to volts
  float mq7AoVoltage = mq7RawVoltage * voltageDividerMultiplier; //converts the voltage back to what the gas sensor sent itself

  float mq5RawVoltage = (analogRead(mq5Pin) / 4095.0) * espMaxVoltage;
  float mq5AoVoltage = mq5RawVoltage * voltageDividerMultiplier;

  Serial.print("MQ-5: ");
  Serial.print(mq5AoVoltage, 2);
  Serial.print(" V | MQ-7: ");
  Serial.print(mq7AoVoltage, 2);
  Serial.println(" V");

  // Buzzer alarm logic (active-low)
  if (mq7AoVoltage > mq7Threshold || mq5AoVoltage > mq5Threshold) {
    digitalWrite(buzzerPin, LOW);  // ON
  } else {
    digitalWrite(buzzerPin, HIGH);  // OFF
  }

  delay(1000);
}
```

Phase 6

```
#define BLYNK_PRINT Serial

#define BLYNK_TEMPLATE_ID "TMPL272tJazjr"
#define BLYNK_TEMPLATE_NAME "Gas Detector"
#define BLYNK_AUTH_TOKEN "En7gqwXl7BDeRVsSUAQg6jWtHvfRb82l"

#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>

// WiFi credentials
char ssid[] = "Snyman";
char pass[] = "[redacted]";

// Sensor pins
const int mq5Pin = 39;    // MQ-5 analog output to GPIO39
const int buzzerPin = 14;  // Active-low buzzer on GPIO14

// Threshold
const float mq5Threshold = 1.0;

// Voltage conversion factors
const float espMaxVoltage = (5.0 / 16800.0) * 10000.0;
const float voltageDividerMultiplier = (10000.0 + 6800.0) / 10000.0;

// Blynk virtual pins
#define VPIN_MQ5 V1
#define VPIN_BUZZER_SWITCH V2

// Buzzer control variable
bool buzzerEnabled = true;

// Blynk app switch → enable/disable buzzer
BLYNK_WRITE(VPIN_BUZZER_SWITCH) {
 buzzerEnabled = param.asInt();  // 1 = enabled, 0 = disabled
 if (!buzzerEnabled) {
  digitalWrite(buzzerPin, HIGH);  // OFF
 }
}

void setup() {
 Serial.begin(115200);
 analogReadResolution(12);  // ESP32 ADC is 12-bit (0–4095)

 pinMode(buzzerPin, OUTPUT);
 digitalWrite(buzzerPin, HIGH);  // buzzer off (active LOW)

 Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
}

void loop() {
 Blynk.run();

 // Read MQ-5 analog value and convert to voltage
 float mq5RawVoltage = (analogRead(mq5Pin) / 4095.0) * espMaxVoltage;
 float mq5AoVoltage = mq5RawVoltage * voltageDividerMultiplier;
```

```
  // Send to Blynk
  Blynk.virtualWrite(VPIN_MQ5, mq5AoVoltage);

  // Buzzer alarm logic
  if (buzzerEnabled && (mq5AoVoltage > mq5Threshold)) {
    digitalWrite(buzzerPin, LOW);  // ON
    Blynk.logEvent("gas_alert", "MQ-5: Gas level exceeded!");
  } else {
    digitalWrite(buzzerPin, HIGH);  // OFF
  }

  delay(1000);  // update every second
}
```

ESP32 Pinout